



INSTITUTO TECNOLÓGICO DE LA LAGUNA
del Tecnológico Nacional de México
Ingeniería en Sistemas Computacionales



LENGUAJES Y AUTOMATAS I I

PERIODO : Ene – Jun / 2026

GRUPO: “A” 8:00-9:00 a.m.

PROYECTO DE ASIGNATURA

ALUMNO:

23130525 Derek Romo Flores
21130835 Martin Sebastian Gonzalez Chairez
22131364 Sergio Antonio Contreras Guerrero
22130852 Ricardo Garcia Martinez

PROFESOR:

Ing. Luis Fernando Gil Vázquez

Torreón, Coah. a 24 de febrero de 2026

Situación didáctica

TecLag Software es una organización dedicada al desarrollo de lenguajes de programación. Actualmente se encuentra trabajando en la implementación de un nuevo lenguaje denominado **PaytonTK**, el cual es un dialecto del lenguaje Python con la particularidad de ser de tipado estático.

El proyecto del compilador de PaytonTK ya cuenta con la implementación de las etapas de Interfaz de Usuario y Análisis Léxico. Como siguiente fase del desarrollo, la empresa ha dado luz verde a la implementación del **Análisis Sintáctico** mediante un **parser predictivo recursivo**, con la posibilidad de integrar de forma paralela el análisis semántico.

El equipo de ingeniería tiene la responsabilidad de:

- Diseñar los procedimientos correspondientes a cada símbolo no-terminal de la gramática.
- Implementar dichos procedimientos en la clase SintacticoSemantico.
- Incorporar manejo adecuado de errores sintácticos, incluyendo descripción clara y número de línea.
- Validar el funcionamiento del parser mediante pruebas correctas e incorrectas.

El objetivo principal de esta práctica es aplicar los fundamentos teóricos de gramáticas libres de contexto, conjuntos PRIMEROS y análisis descendente LL(1), para desarrollar un analizador sintáctico funcional y correctamente estructurado.

Analisis

Definición de Parser Predictivo

Un Parser Predictivo Recursivo es un tipo de analizador sintáctico descendente que construye la derivación más a la izquierda de una cadena de entrada, procesándola de izquierda a derecha.

Este tipo de parser:

- Implementa un procedimiento por cada símbolo no-terminal de la gramática.
- Utiliza los conjuntos PRIMEROS para decidir qué producción aplicar.
- No requiere retroceso cuando la gramática es LL(1).
- Emplea una variable de preanálisis (lookahead) para tomar decisiones sintácticas.

En la implementación, cada no-terminal de la gramática del lenguaje PaytonTK se traduce en un procedimiento recursivo dentro de la clase **SintacticoSemantico**.

Conjuntos PRIMEROS

A continuación, se presentan los conjuntos PRIMEROS calculados a partir de la gramática oficial de PaytonTK:

PRIMERO(PROGRAMA)

PROGRAMA $\rightarrow$ INSTRUCCION PROGRAMA | ϵ

PRIMERO(PROGRAMA) = { def, int, float, string, id, if, while, print, ϵ }

PRIMERO(INSTRUCCION)

INSTRUCCION $\rightarrow$ FUNCION | PROPOSICION

PRIMERO(INSTRUCCION) = { def, int, float, string, id, if, while, print }

PRIMERO(FUNCION)

FUNCION $\rightarrow$ def id (ARGUMENTOS) : TIPO_RETORNO PROPOSICIONES_OPTATIVAS return RESULTADO ::

PRIMERO(FUNCION) = { def }

PRIMERO(PROPOSICION)

PROPOSICION $\rightarrow$ DECLARACION_VARS | ASIGNACION | IF | WHILE | LLAMADA_FUNC | PRINT

PRIMERO(PROPOSICION) = { int, float, string, id, if, while, print }

PRIMERO(DECLARACION_VARS)

DECLARACION_VARS $\rightarrow$ TIPO_DATO id DECLARACION_VARS'

PRIMERO(DECLARACION_VARS) = { int, float, string }

PRIMERO(TIPO_DATO)

PRIMERO(TIPO_DATO) = { int, float, string }

PRIMERO(TIPO_RETORNO)

PRIMERO(TIPO_RETORNO) = { void, int, float, string }

PRIMERO(RESULTADO)

RESULTADO → EXPRESION | void

PRIMERO(RESULTADO) = { id, num, num.num, (, literal, void }

PRIMERO(EXPRESION)

PRIMERO(EXPRESION) = { id, num, num.num, (, literal }

PRIMERO(TERMINO)

PRIMERO(TERMINO) = { id, num, num.num, (}

PRIMERO(FACTOR)

PRIMERO(FACTOR) = { id, num, num.num, (}

PRIMERO(PROPOSICIONES_OPTATIVAS)

PROPOSICIONES_OPTATIVAS → PROPOSICION PROPOSICIONES_OPTATIVAS | ε

PRIMERO(PROPOSICIONES_OPTATIVAS) = { int, float, string, id, if, while, print, ε }

Diseño**Pseudocódigo de los Procedimientos**

Cada procedimiento incluye el nombre de su autor y la producción que analiza.

//Autor: Sergio Antonio Contreras Guerrero

```
-----Procedure PROGRAMA-----
Begin
  if preAnalisis == "def " then
    begin
      // PROGRAMA -> INSTRUCCION PROGRAMA
      INTRODUCCION()
      PROGRAMA()
    end
  else if preAnalisis == "{ id, if, while, print, int, float, string }"
    begin
```

```

        // PROGRAMA -> INSTRUCCION PROGRAMA
        INTRODUCCION()
        PROGRAMA()
    else
        begin
            //programa -> empty
        end
    end
end

-----Procedure INSTRUCCION-----
Begin
    If preAnalisis == " def" then
        begin
            //INSTRUCCION -> FUNCION
            FUNCION()
        end
    Else if preAnalisis == "{ id, if, while, print, int, float, string }" then
        begin
            //INSTRUCCION -> PROPOSICION
            PROPOSICION()
        end
    end
End

-----Procedure FUNCION-----

Begin

    if preAnalisis == " def" then

        begin

            // FUNCION → def id ( ARGUMENTOS ) : TIPO_RETORNO

            //PROPOSICIONES_OPTATIVAS

            emparejar( "def" )

            emparejar( "id" )

            emparejar( "(" )

            ARGUMENTOS()

            emparejar( ")" )

            emparejar( ":" )

            TIPO_RETORNO()

            PROPOSICIONES_OPTATIVAS()

            emparejar( "return" )

            RESULTADO ()

        end

    end

end

```

```
-----Procedure ARGUMENTOS-----

Begin

    if preAnalisis == " int, float, string " then

        begin

            // ARGUMENTOS -> TIPO_DATO id ARGUMENTOS'

            TIPO_DATO()

            emparejar( "id" )

            ARGUMENTOS' ()

        end

    else

        begin

            //ARGUMENTOS -> empty

        end

    end

end

-----Procedure ARGUMENTOS'-----

Begin

    if preAnalisis == " , " then

        begin

            // ARGUMENTOS -> , TIPO_DATO id ARGUMENTOS'

            emparejar( " , " )

            TIPO_DATO()

            emparejar( "id" )

            ARGUMENTOS' ()

        end

    end

end
```

```
    else
        begin
            //ARGUMENTOS -> empty
        end
    end
end
```

//Autor: Martín Sebastian Gonzalez Chairez

```
-----Procedure DECLARACION_VARS-----

Begin
    if preAnalisis IN {int, float, string} then
        Begin
            TIPO_DATO ()
            emparejar ("id")
            DECLARACION_VARS' ()
        End
    else
        Begin
            error ()
        End
    End
End
```

```
-----Procedure DECLARACION_VARS'-----

Begin
    if preAnalisis == "," then
        Begin
            emparejar (",")
            emparejar ("id")
            DECLARACION_VARS' ()
        End
    else
```

```
Begin
    // DECLARACION_VARS' -> empty
End
End
```

-----**Procedure TIPO_RETORNO**-----

```
Begin
    if preAnalisis == "void" then
        Begin
            emparejar ("void")
        End
    else if preAnalisis IN {int, float, string} then
        Begin
            //TIPO_RETORNO -> TIPO_DATO
            TIPO_DATO ()
        End
    else
        Begin
            error ()
        End
    End
End
```

-----**Procedure TIPO_DATO**-----

```
Begin
    if preAnalisis == "int" then
        Begin
            emparejar ("int")
        End
    else if preAnalisis == "float" then
```



```
        Begin
            emparejar ("float")
        End
    else if preAnalisis == "string" then
        Begin
            emparejar ("string")
        End
    else
        Begin
            error ()
        End
    End
```

-----**Procedure RESULTADO**-----

```
Begin
    if preAnalisis IN {id, num, num.num, (, literal} then
        Begin
            // RESULTADO -> EXPRESION
            EXPRESION ()
        End
    else if preAnalisis == "void" then
        Begin
            //RESULTADO -> void
            emparejar ( "void" )
        End
    else
        Begin
            error ()
        End
    End
```

End

-----**Procedure** PROPOSICIONES_OPTATIVAS-----

Begin

if preAnalisis IN {int, float, string, id, if, while, print}

Begin

PROPOSICION ()

PROPOSICIONES_OPTATIVAS ()

End

else

Begin

// PROPOSICIONES_OPTATIVAS -> empty

End

End

//Autor: Derek Romo Flores

```
-----Procedure PROPOSICION-----  
  
Begin  
  
    if preAnalisis IN {int, float, string} then  
  
        begin  
  
            // PROPOSICION → DECLARACION_VARS  
  
            DECLARACION_VARS()  
  
        end  
  
  
    else if preAnalisis == id then  
  
        begin  
  
            // PROPOSICION → id PROPOSICION'  
  
            emparejar(id)  
  
            PROPOSICION_PRIMA()  
  
        end  
  
  
    else if preAnalisis == if then  
  
        begin  
  
            // PROPOSICION → if CONDICION : PROPOSICIONES_OPTATIVAS else : PROPOSICIONES_OPTATIVAS  
::  
  
            emparejar(if)  
  
            CONDICION()  
  
            emparejar(":")  
  
            PROPOSICIONES_OPTATIVAS()  
  
            emparejar(else)  
  
            emparejar(":")  
  
            PROPOSICIONES_OPTATIVAS()  
  
            emparejar("::")  
  
        end  
  
    end
```

```
else if preAnálisis == while then
begin
    // PROPOSICION → while CONDICION : PROPOSICIONES_OPTATIVAS ::
    emparejar(while)
    CONDICION()
    emparejar(":")
    PROPOSICIONES_OPTATIVAS()
    emparejar("::")
end

else if preAnálisis == print then
begin
    // PROPOSICION → print ( EXPRESION )
    emparejar(print)
    emparejar("(")
    EXPRESION()
    emparejar(")")
end

else
    error("PROPOSICION: estructura no válida en línea " + numLinea)
End
```

```
-----Procedure PROPOSICION_PRIMA-----  
  
Begin  
  
    if preAnalisis == opasig then  
  
        begin  
  
            // PROPOSICION' → opasig EXPRESION  
  
            emparejar(opasig)  
  
            EXPRESION()  
  
        end  
  
    else if preAnalisis == "(" then  
  
        begin  
  
            // PROPOSICION' → ( LISTA_EXPRESIONES )  
  
            emparejar("(")  
  
            LISTA_EXPRESIONES()  
  
            emparejar(")")  
  
        end  
  
    else  
  
        error("PROPOSICION_PRIMA: se esperaba asignación o llamada a función en línea " +  
numLinea)  
  
    End  
  
-----Procedure LISTA_EXPRESIONES-----  
  
Begin  
  
    if preAnalisis IN {id, num, num.num, "(", literal} then  
  
        begin  
  
            // LISTA_EXPRESIONES → EXPRESION LISTA_EXPRESIONES'  
  
            EXPRESION()  
  
            LISTA_EXPRESIONES_PRIMA()  
  
        end  
  
    else
```

```
begin
    // LISTA_EXPRESIONES → empty
end
End
```

-----**Procedure LISTA_EXPRESIONES_PRIMA**-----

```
Begin
    if preAnalisis == "," then
        begin
            // LISTA_EXPRESIONES' → , EXPRESION LISTA_EXPRESIONES'
            emparejar(",")
            EXPRESION()
            LISTA_EXPRESIONES_PRIMA()
        end
    else
        begin
            // LISTA_EXPRESIONES' → empty
        end
    end
End
```

-----**Procedure CONDICION**-----

```
Begin
    if preAnalisis IN {id, num, num.num, "(", literal} then
        begin
            // CONDICION → EXPRESION oprel EXPRESION
            EXPRESION()
            emparejar(oprel)
            EXPRESION()
        end
    end
```

```

else

    error("CONDICION: condición inválida en línea " + numLinea)

End

```

```

-----Procedure EXPRESION-----
Begin
    if preAnalisis IN {"id", "num", "num.num", "("} then
        begin
            // EXPRESION -> TERMINO EXPRESION'
            TERMINO()
            EXPRESION()
        End
    else if preAnalisis IN {"literal"} then
        begin
            // EXPRESION -> literal
            Emparejar("literal")
        End
    else
        begin
            error //Error de tipo sintactico
        end
    end
end

```

```

-----Procedure EXPRESION'-----
Begin
    if preAnalisis IN {"opsuma"} then
        begin
            // EXPRESION' -> opsuma TERMINO EXPRESION'
            Emparejar("opsuma")
            TERMINO()
            EXPRESIONP()
        End
    else
        begin
            // EXPRESION' -> empty
        end
    end
end

```

```

-----Procedure TERMINO-----
Begin
    if preAnalisis IN {"id", "num", "num.num", "("} then
        begin
            // TERMINO -> FACTOR TERMINO'
            FACTOR()
            TERMINO()
        End
    else
        begin
            error //Error de tipo sintactico
        end
    end
end

```

```

-----Procedure TERMINO'-----
Begin
    if preAnalisis IN {"opmult"} then
        begin
            // TERMINO' -> opmult FACTOR TERMINO'
            Emparejar("opmult")
            FACTOR()
        End
    end
end

```

```

        TERMINO()
    End
    else
    begin
        // TERMINO' -> empty
    end
end

```

```

-----Procedure FACTOR-----
Begin
    if preAnalisis IN {"id"} then
    begin
        // FACTOR -> id FACTOR'
        Emparejar("id")
        FACTORP()
    End
    else if preAnalisis IN {"num"} then
    begin
        // FACTOR -> num
        Emparejar("num")
    End
    else if preAnalisis IN {"num.num"} then
    begin
        // FACTOR -> num.num
        Emparejar("num.num")
    End
    else if preAnalisis IN {"("} then
    begin
        // FACTOR -> ( EXPRESION )
        Emparejar("(")
        EXPRESION()
        Emparejar(")")
    End
    else
    begin
        error    //Error de tipo sintactico
    end
end

```

```

PRIMERO ( FACTOR' ) = { (, empty }

```

```

-----Procedure FACTOR'-----
Begin
    if preAnalisis IN {"("} then
    begin
        // FACTOR' -> ( LISTA_EXPRESIONES )
        Emparejar("(")
        LISTA_EXPRESIONES()
        Emparejar(")")
    end
    else
    begin
        // FACTOR' -> empty
    end
end

```


Diagrama UML

SintacticoSemantico

```
-----  
- cmp : Compilador  
- analizarSemantica : boolean  
- preAnalisis : String  
-----  
+ SintacticoSemantico(c : Compilador)  
+ analizar(analizarSemantica : boolean) : void  
  
- emparejar(t : String) : void  
- errorEmparejar(_token : String, _lexema : String, numLinea : int) : void  
- error(_descripError : String) : void  
  
- PROGRAMA() : void  
- INSTRUCCION() : void  
- FUNCION() : void  
- ARGUMENTOS() : void  
- ARGUMENTOS_2() : void  
  
- DECLARACION_VARS() : void  
- DECLARACION_VARS_2() : void  
- TIPO_DATO() : void  
- TIPO_RETORNO() : void  
- RESULTADO() : void  
- PROPOSICIONES_OPTATIVAS() : void  
  
- PROPOSICION() : void  
- PROPOSICION_2() : void  
- LISTA_EXPRESIONES() : void  
- LISTA_EXPRESIONES_2() : void  
- CONDICION() : void  
  
- EXPRESION() : void  
- EXPRESION_2() : void  
- TERMINO() : void  
- TERMINO_2() : void  
- FACTOR() : void  
- FACTOR_2() : void  
-----
```

Código

SintacticoSemantico.JAVA

```

/*-----
*:
*:          INSTITUTO TECNOLOGICO DE LA LAGUNA
*:          INGENIERIA EN SISTEMAS COMPUTACIONALES
*:          LENGUAJES Y AUTOMATAS II
*:
*:          SEMESTRE: __ENE/JUN__      HORA: __8:00 - 9:00__ HRS
*:
*:
*:          Clase con la funcionalidad del Analizador Sintactico
*
*:
*:
*: Archivo      : SintacticoSemantico.java
*: Autor        : Fernando Gil ( Estructura general de la clase )
*:              Grupo de Lenguajes y Automatas II ( Procedures )
*: Fecha        : 03/SEP/2014
*: Compilador   : Java JDK 7
*: Descripción  : Esta clase implementa un parser descendente del tipo
*:              Predictivo Recursivo. Se forma por un metodo por cada simbolo
*:              No-Terminal de la gramatica mas el metodo emparejar ().
*:              El analisis empieza invocando al metodo del simbolo inicial.
*: Ult.Modif.   :
*: Fecha        Modificó          Modificacion
*:=====
*: 22/Feb/2015 FGil                -Se mejoro errorEmparejar () para mostrar el
*:                                numero de linea en el codigo fuente donde
*:                                ocurrio el error.
*: 08/Sep/2015 FGil                -Se dejo lista para iniciar un nuevo analizador
*:                                sintactico.
*: 20/FEB/2023 F.Gil, Oswi         -Se implementaron los procedures del parser
*:                                predictivo recursivo de leng BasicTec.
*:-----
*/
package compilador;

import javax.swing.JOptionPane;

public class SintacticoSemantico {

    private Compilador cmp;
    private boolean    analizarSemantica = false;
    private String     preAnalisis;

    //-----
    // Constructor de la clase, recibe la referencia de la clase principal del
    // compilador.
    //
    public SintacticoSemantico(Compilador c) {
        cmp = c;
    }

    //-----
    //-----
    // Metodo que inicia la ejecucion del analisis sintactico predictivo.
    // analizarSemantica : true = realiza el analisis semantico a la par del sintactico
    //                   false= realiza solo el analisis sintactico sin comprobacion semantica
    public void analizar(boolean analizarSemantica) {
        this.analizarSemantica = analizarSemantica;
        preAnalisis = cmp.be.preAnalisis.complex;

        // * * * INVOCAR AQUI EL PROCEDURE DEL SIMBOLO INICIAL * * *
        PROGRAMA();
    }

    //-----

    private void emparejar(String t) {
        if (cmp.be.preAnalisis.complex.equals(t)) {
            cmp.be.siguiente();
        }
    }
}

```

```

        preAnalisis = cmp.be.preAnalisis.complex;
    } else {
        errorEmparejar( t, cmp.be.preAnalisis.lexema, cmp.be.preAnalisis.numLinea );
    }
}

//-----
// Metodo para devolver un error al emparejar
//-----

private void errorEmparejar(String _token, String _lexema, int numLinea ) {
    String msjError = "";

    if (_token.equals("id")) {
        msjError += "Se esperaba un identificador";
    } else if (_token.equals("num")) {
        msjError += "Se esperaba una constante entera";
    } else if (_token.equals("num.num")) {
        msjError += "Se esperaba una constante real";
    } else if (_token.equals("literal")) {
        msjError += "Se esperaba una literal";
    } else if (_token.equals("oparit")) {
        msjError += "Se esperaba un operador aritmetico";
    } else if (_token.equals("oprel")) {
        msjError += "Se esperaba un operador relacional";
    } else if (_token.equals("opasig")) {
        msjError += "Se esperaba operador de asignacion";
    } else {
        msjError += "Se esperaba " + _token;
    }
    msjError += " se encontró " + ( _lexema.equals ( "$" )? "fin de archivo" : _lexema ) +
        ". Linea " + numLinea;          // FGil: Se agregó el numero de línea

    cmp.me.error(Compilador.ERR_SINTACTICO, msjError);

    // Avanzar token para evitar ciclo infinito
    cmp.be.siguiente();
    preAnalisis = cmp.be.preAnalisis.complex;
}

// Fin de ErrorEmparejar
//-----
// Metodo para mostrar un error sintactico

private void error(String _descripcionError) {
    System.out.println("Token actual: " + preAnalisis); //Agregado por Sergio
    cmp.me.error(cmp.ERR_SINTACTICO, _descripcionError);

    // Avanzar token para evitar ciclo infinito (Agregado por Sergio)
    cmp.be.siguiente();
    preAnalisis = cmp.be.preAnalisis.complex;
}

// Fin de error
//-----
// * * * * PEGAR AQUI EL CODIGO DE LOS PROCEDURES * * * *
//-----

// =====
// ===== Sergio =====
// =====

// ----- Procedure 1 -----
private void PROGRAMA() {
    if (preAnalisis.equals("def") || preAnalisis.equals("id") || preAnalisis.equals("if")
        || preAnalisis.equals("while") || preAnalisis.equals("print") || preAnalisis.equals("int")
        || preAnalisis.equals("float") || preAnalisis.equals("string")) {
        // PROGRAMA -> INSTRUCCION PROGRAMA
        INSTRUCCION();
        PROGRAMA();
    } else {
        //PROGRAMA -> empty
    }
}
}

```

```

// ----- Procedure 2 -----
private void INSTRUCCION() {
    if (preAnalisis.equals("def")) {
        //INSTRUCCION -> FUNCION
        FUNCION();
    } else if (preAnalisis.equals("id") || preAnalisis.equals("if")
        || preAnalisis.equals("while") || preAnalisis.equals("print")
        || preAnalisis.equals("int") || preAnalisis.equals("float")
        || preAnalisis.equals("string")) {

        //INSTRUCCION -> PROPOSICION
        PROPOSICION();
    } else {
        error("Se esperaba una instruccion o funcion");
    }
}

// ----- Procedure 3 -----
private void FUNCION() {
    if (preAnalisis.equals("def")) {
        // FUNCION -> def id ( ARGUMENTOS ) : TIPO_RETORNO PROPOSICIONES_OPTATIVAS return RESULTADO
        emparejar("def");
        emparejar("id");
        emparejar("(");
        ARGUMENTOS();
        emparejar(")");
        emparejar(":");
        TIPO_RETORNO();
        PROPOSICIONES_OPTATIVAS();

        // <- FALTABA ESTA PARTE:
        emparejar("return");
        RESULTADO();

    } else {
        error("Se esperaba la definicion de una funcion");
    }
}

// ----- Procedure 4 -----
private void ARGUMENTOS() {
    if (preAnalisis.equals("int")
        || preAnalisis.equals("float") || preAnalisis.equals("string")) {
        // ARGUMENTOS -> TIPO_DATO id ARGUMENTOS'
        TIPO_DATO();
        emparejar("id");
        ARGUMENTOS_2();
    } else {
        //ARGUMENTOS -> empty
    }
}

// ----- Procedure 5 -----
private void ARGUMENTOS_2() {
    if (preAnalisis.equals(",")) {
        // ARGUMENTOS -> , TIPO_DATO id ARGUMENTOS'
        emparejar(",");
        TIPO_DATO();
        emparejar("id");
        ARGUMENTOS_2();
    } else {
        //ARGUMENTOS_2 -> empty
    }
}

// =====
// ===== Sebas =====
// =====

// ----- Procedure 6 -----
private void DECLARACION_VARS() {
    if (preAnalisis.equals("int")
        || preAnalisis.equals("float")
        || preAnalisis.equals("string")) {

```

```

        // DECLARACION_VARS -> TIPO_DATO id DECLARACION_VARS'
        TIPO_DATO();
        emparejar("id");
        DECLARACION_VARS_2();
    } else {
        error("Declaracion de variables invalida");
    }
}

// ----- Procedure 7 -----
private void DECLARACION_VARS_2() {
    if (preAnalisis.equals(",")) {
        // DECLARACION_VARS' -> , id DECLARACION_VARS'
        emparejar(",");
        emparejar("id");
        DECLARACION_VARS_2();
    } else {
        // DECLARACION_VARS_2 -> empty
    }
}

// ----- Procedure 8 -----
private void TIPO_DATO() {
    if (preAnalisis.equals("int")) {
        emparejar("int");
    } else if (preAnalisis.equals("float")) {
        emparejar("float");
    } else if (preAnalisis.equals("string")) {
        emparejar("string");
    } else {
        error("Tipo de dato invalido");
    }
}

// ----- Procedure 9 -----
private void TIPO_RETORNO() {
    if (preAnalisis.equals("void")){
        emparejar ("void");
    }else if (preAnalisis.equals("int") ||
        preAnalisis.equals("float") ||
        preAnalisis.equals("string")){
        // TIPO_RETORNO -> TIPO_DATO
        TIPO_DATO();
    }else{
        error ("Se esperaba un tipo de retorno");
    }
}

// ----- Procedure 10 -----
private void RESULTADO(){
    if (preAnalisis.equals("id") ||
        preAnalisis.equals("num") ||
        preAnalisis.equals("num.num") ||
        preAnalisis.equals("(") ||
        preAnalisis.equals("literal")){

        //RESULTADO -> EXPRESION
        EXPRESION ();
    }else if (preAnalisis.equals("void")){
        //RESULTADO -> void
        emparejar ("void");
    }else{
        error ("Se esperaba un resultado");
    }
}

// ----- Procedure 11 -----
private void PROPOSICIONES_OPTATIVAS(){
    if (preAnalisis.equals("int") ||
        preAnalisis.equals("float") ||
        preAnalisis.equals("string") ||
        preAnalisis.equals("id") ||
        preAnalisis.equals("if") ||
        preAnalisis.equals("while") ||

```

```

        preAnalisis.equals("print")){

            //PROPOSICIONES_OPTATIVAS -> PROPOSICION PROPOSICIONES_OPTATIVAS
            PROPOSICION ();
            PROPOSICIONES_OPTATIVAS ();
        }else{
            //PROPOSICIONES_OPTATIVAS -> empty
        }
    }

// =====
// ===== Derek =====
// =====

// ----- Procedure 12 -----
private void PROPOSICION() {

    if (preAnalisis.equals("int")
        || preAnalisis.equals("float")
        || preAnalisis.equals("string")) {

        // PROPOSICION -> DECLARACION_VARS
        DECLARACION_VARS ();
    } else if (preAnalisis.equals("id")) {
        // PROPOSICION -> id PROPOSICION'
        emparejar("id");
        PROPOSICION_2 ();
    } else if (preAnalisis.equals("if")) {
        // PROPOSICION -> if CONDICION : PROPOSICIONES_OPTATIVAS else : PROPOSICIONES_OPTATIVAS ::
        emparejar("if");
        CONDICION();
        emparejar(":");
        PROPOSICIONES_OPTATIVAS ();
        emparejar("else");
        emparejar(":");
        PROPOSICIONES_OPTATIVAS ();
        emparejar("::");
    } else if (preAnalisis.equals("while")) {
        // PROPOSICION -> while CONDICION : PROPOSICIONES_OPTATIVAS ::
        emparejar("while");
        CONDICION();
        emparejar(":");
        PROPOSICIONES_OPTATIVAS ();
        emparejar("::");
    } else if (preAnalisis.equals("print")) {
        // PROPOSICION -> print ( EXPRESION )
        emparejar("print");
        emparejar("(");
        EXPRESION();
        emparejar(")");
    } else {
        error("Proposicion invalida");
    }
}

// ----- Procedure 13 -----
private void PROPOSICION_2() {

    if (preAnalisis.equals("opasig")) {
        // PROPOSICION' -> opasig EXPRESION
        emparejar("opasig");
        EXPRESION();
    } else if (preAnalisis.equals("(")) {
        // PROPOSICION' -> ( LISTA_EXPRESIONES )
        emparejar("(");
        LISTA_EXPRESIONES ();
        emparejar(")");
    } else {
        error("Se esperaba asignacion o llamada a funcion");
    }
}

// ----- Procedure 14 -----
private void LISTA_EXPRESIONES() {

```

```

        if (preAnalisis.equals("id")
            || preAnalisis.equals("num")
            || preAnalisis.equals("num.num")
            || preAnalisis.equals("(")
            || preAnalisis.equals("literal")) {

            // LISTA_EXPRESIONES -> EXPRESION LISTA_EXPRESIONES'
            EXPRESION();
            LISTA_EXPRESIONES_2();
        } else {
            // LISTA_EXPRESIONES -> empty
        }
    }

// ----- Procedure 15 -----
private void LISTA_EXPRESIONES_2() {

    if (preAnalisis.equals(",")) {
        // LISTA_EXPRESIONES' -> , EXPRESION LISTA_EXPRESIONES'
        emparejar(",");
        EXPRESION();
        LISTA_EXPRESIONES_2();
    } else {
        // LISTA_EXPRESIONES_2 -> empty
    }
}

// ----- Procedure 16 -----
private void CONDICION() {

    if (preAnalisis.equals("id")
        || preAnalisis.equals("num")
        || preAnalisis.equals("num.num")
        || preAnalisis.equals("(")
        || preAnalisis.equals("literal")) {

        // CONDICION -> EXPRESION oprel EXPRESION
        EXPRESION();
        emparejar("oprel");
        EXPRESION();
    } else {
        error("Condicion invalida");
    }
}

// =====
// ===== Ricardo =====
// =====

// ----- Procedure 17 -----
private void EXPRESION() { //CORRECCION DE MI BRO: Corrijo la recursión infinita manteniendo tu estilo:

    if (preAnalisis.equals("id")
        || preAnalisis.equals("num")
        || preAnalisis.equals("num.num")
        || preAnalisis.equals("(")) {

        // EXPRESION -> TERMINO EXPRESION'
        TERMINO();
        EXPRESION_2();
    } else if (preAnalisis.equals("literal")) {
        // EXPRESION -> literal
        emparejar("literal");
    } else {
        error("Expresion invalida");
    }
}

// ----- Procedure 18 -----
private void EXPRESION_2() {

    if (preAnalisis.equals("opsuma")) {
        // EXPRESION' -> opsuma TERMINO EXPRESION'

```

```

        emparejar("opsuma");
        TERMINO();
        EXPRESION_2();
    } else {
        // EXPRESION_2 -> empty
    }
}

// ----- Procedure 19 -----
private void TERMINO() {
    if (preAnalisis.equals("id"))
        || preAnalisis.equals("num")
        || preAnalisis.equals("num.num")
        || preAnalisis.equals("(") {

        // TERMINO -> FACTOR TERMINO'
        FACTOR();
        TERMINO_2();
    } else {
        error("Termino invalido");
    }
}

// ----- Procedure 20 -----
private void TERMINO_2() {
    if (preAnalisis.equals("opmult")) {
        // TERMINO' -> opmult FACTOR TERMINO'
        emparejar("opmult");
        FACTOR();
        TERMINO_2();
    } else {
        // TERMINO_2 -> empty
    }
}

// ----- Procedure 21 -----
private void FACTOR() {
    if (preAnalisis.equals("id")) {
        // FACTOR -> id FACTOR'
        emparejar("id");
        FACTOR_2();
    } else if (preAnalisis.equals("num")) {
        emparejar("num");
    } else if (preAnalisis.equals("num.num")) {
        emparejar("num.num");
    } else if (preAnalisis.equals("(")) {
        emparejar("(");
        EXPRESION();
        emparejar(")");
    } else {
        error("Factor invalido");
    }
}

// ----- Procedure 22 -----
private void FACTOR_2() {
    if (preAnalisis.equals("(")) {
        // FACTOR' -> ( LISTA_EXPRESIONES )
        emparejar("(");
        LISTA_EXPRESIONES();
        emparejar(")");
    } else {
        // FACTOR_2 -> empty
    }
}
}

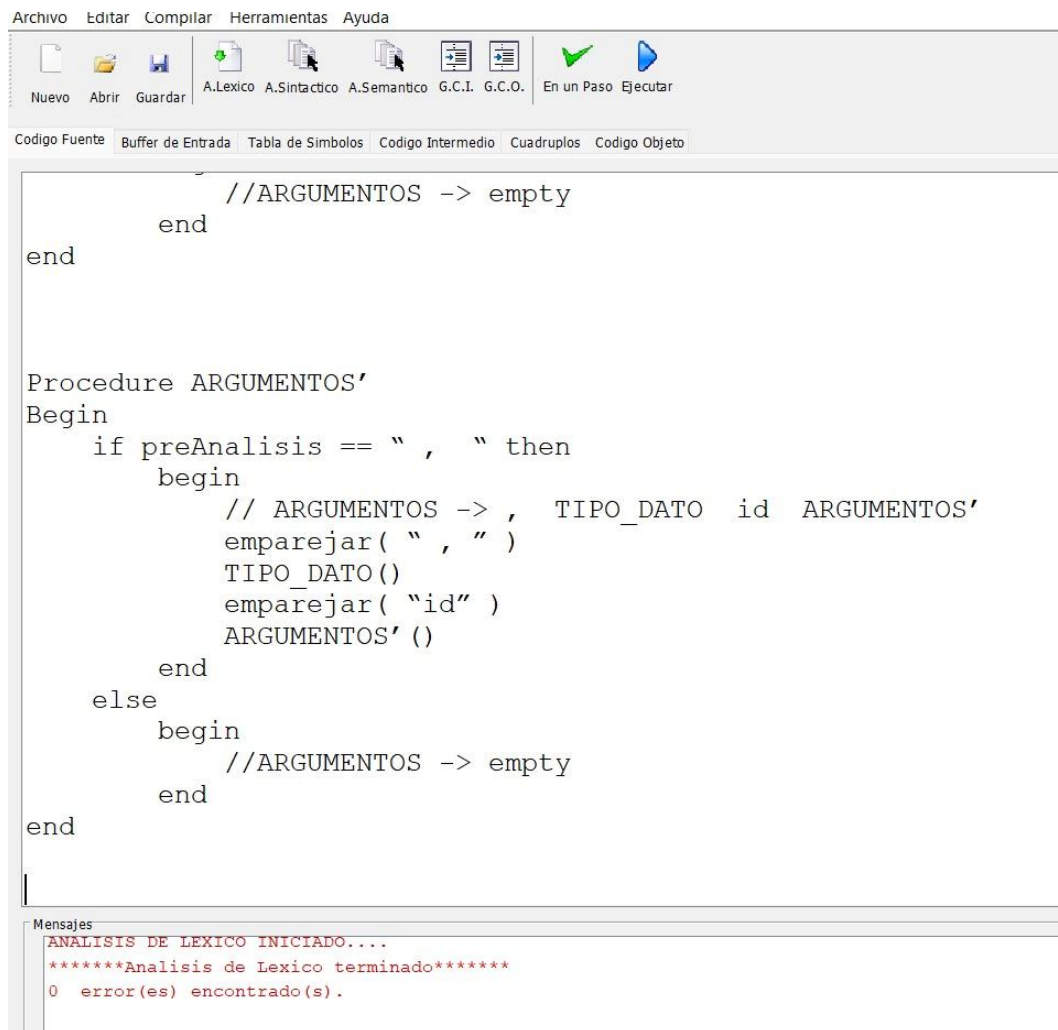
```


Prueba de Ejecución

Prueba 1 – Análisis Léxico Correcto

Descripción:

En esta prueba se ejecuta únicamente el análisis léxico del archivo fuente que contiene la definición de los procedimientos del parser.



The screenshot shows a software interface with a menu bar (Archivo, Editar, Compilar, Herramientas, Ayuda) and a toolbar with icons for file operations and analysis steps. The main window displays a code editor with the following text:

```
        //ARGUMENTOS -> empty
    end
end

Procedure ARGUMENTOS'
Begin
    if preAnalisis == " , " then
        begin
            // ARGUMENTOS -> , TIPO_DATO id ARGUMENTOS'
            emparejar( " , " )
            TIPO_DATO()
            emparejar( "id" )
            ARGUMENTOS' ()
        end
    else
        begin
            //ARGUMENTOS -> empty
        end
    end
end
```

At the bottom, a 'Mensajes' (Messages) window displays the following output:

```
ANALISIS DE LEXICO INICIADO....
*****Analisis de Lexico terminado*****
0 error(es) encontrado(s).
```

Prueba 2 – Verificación en Tabla de Símbolos

Descripción:

En esta prueba se inspecciona la tabla de símbolos generada después del análisis léxico.

Nuevo

Abrir

Guardar

A.Lexico

A.Sintactico

A.Semantico

G.C.I.

G.C.O.

En un Paso

Ejecutar

Codigo Fuente

Buffer de Entrada

Tabla de Simbolos

Codigo Intermedio

Cuadрупlos

Codigo Objeto

Complex	Lexema	Entrada
id	PROGRAMA	1
id	INSTRUCCION	2
id	PROGRAMA	1
id	PRIMERO	3
(	(	0
id	PROGRAMA	1
)	)	0
opasig	=	0
id	PRIMEROS	4
(	(	0
id	INSTRUCCION	2
)	)	0
id	U	5
{	{	0
id	empty	6
}	}	0
id	INSTRUCCION	2
id	FUNCION	7
id	PROPOSICION	8
id	PRIMERO	3
(	(	0
id	INSTRUCCION	2
)	)	0
opasig	=	0
id	PRIMEROS	4
(	(	0
id	FUNCION	7
)	)	0
id	U	5
id	INSTRUCCION	2
id	FUNCION	7
id	PROPOSICION	8
id	PRIMEROS	4
(	(	0
id	PROPOSICION	8
)	)	0
id	FUNCION	7
def	def	0
id	id	9

Nuevo Abrir Guardar Nuevo Asignado Asignado Ver Ver En el Paso Guardar		
Codigo Fuente	Buffer de Entrada	Tabla de Simbolos Codigo Intermedio Cuadрупlos Codigo Objeto
Complex	Lexema	Entrada
(	(	0
id	ARGUMENTOS	10
)	)	0
:	:	0
id	TIPO_RETORNO	11
id	PROPOSICIONES_OPTATIVAS	12
return	return	0
id	RESULTADO	13
:	:	0
:	:	0
id	PRIMERO	3
(	(	0
id	FUNCION	7
)	)	0
opasig	=	0
{	{	0
def	def	0
}	}	0
id	ARGUMENTOS	10
id	TIPO_DATO	14
id	id	9
id	ARGUMENTOS	10
id	PRIMERO	3
(	(	0
id	ARGUMENTOS	10
)	)	0
opasig	=	0
id	PRIMEROS	4
(	(	0
id	TIPO_DATO	14
)	)	0
id	U	5
{	{	0
id	empty	6
}	}	0
id	ARGUMENTOS	10
,	,	0
id	TIPO_DATO	14
id	id	9
id	ARGUMENTOS	10

Nuevo

Abrir

Guardar

A.Lexico

A.Sintactico

A.Semantico

G.C.I.

G.C.O.

En un Paso

Ejecutar

Codigo Fuente

Buffer de Entrada

Tabla de Simbolos

Codigo Intermedio

Cuadрупlos

Codigo Objeto

Complex	Lexema	Entrada
def	def	0
id	then	19
id	begin	20
opmult	/	0
opmult	/	0
id	PROGRAMA	1
signo	-	0
oprel	>	0
id	INS TRUCCION	2
id	PROGRAMA	1
id	INTRODUCCION	21
(	(	0
)	)	0
id	PROGRAMA	1
(	(	0
)	)	0
id	end	22
else	else	0
if	if	0
id	preAnalisis	18
oprel	==	0
{	{	0
id	id	9
,	,	0
if	if	0
,	,	0
while	while	0
,	,	0
print	print	0
,	,	0
int	int	0
,	,	0
float	float	0
,	,	0
string	string	0
}	}	0
id	begin	20
opmult	/	0
opmult	/	0

Nuevo

Abrir

Guardar

A.Lexico

A.Sintactico

A.Semantico

G.C.I.

G.C.O.

En un Paso

Ejecutar

Codigo Fuente

Buffer de Entrada

Tabla de Simbolos

Codigo Intermedio

Cuadрупlos

Codigo Objeto

Complex	Lexema	Entrada
}	}	0
id	begin	20
opmult	/	0
opmult	/	0
id	PROGRAMA	1
signo	-	0
oprel	>	0
id	INSTRUCCION	2
id	PROGRAMA	1
id	INTRODUCCION	21
(	(	0
)	)	0
id	PROGRAMA	1
(	(	0
)	)	0
else	else	0
id	begin	20
opmult	/	0
opmult	/	0
id	programa	23
signo	-	0
oprel	>	0
id	empty	6
id	end	22
id	end	22
id	Procedure	16
id	INSTRUCCION	2
id	Begin	17
id	If	24
id	preAnalysis	18
oprel	==	0
def	def	0
id	then	19
id	begin	20
opmult	/	0
opmult	/	0
id	INSTRUCCION	2
signo	-	0
oprel	>	0
id	FUNCION	7

Nuevo

Abrir

Guardar

A.Lexico

A.Sintactico

A.Semantico

G.C.I.

G.C.O.

En un Paso

Ejecutar

Codigo Fuente

Buffer de Entrada

Tabla de Simbolos

Codigo Intermedio

Cuadрупlos

Codigo Objeto

Complex	Lexema	Entrada
}	}	0
id	begin	20
opmult	/	0
opmult	/	0
id	PROGRAMA	1
signo	-	0
oprel	>	0
id	INSTRUCCION	2
id	PROGRAMA	1
id	INTRODUCCION	21
(	(	0
)	)	0
id	PROGRAMA	1
(	(	0
)	)	0
else	else	0
id	begin	20
opmult	/	0
opmult	/	0
id	programa	23
signo	-	0
oprel	>	0
id	empty	6
id	end	22
id	end	22
id	Procedure	16
id	INSTRUCCION	2
id	Begin	17
id	If	24
id	preAnalysis	18
oprel	==	0
def	def	0
id	then	19
id	begin	20
opmult	/	0
opmult	/	0
id	INSTRUCCION	2
signo	-	0
oprel	>	0
id	FUNCION	7

Prueba 3 de Ejecución – Análisis Sintáctico Correcto**Descripcion:**

En esta prueba se ejecutó el módulo de **Análisis Sintáctico** del compilador utilizando el parser predictivo recursivo implementado en la clase SintacticoSemantico.

```
mensajes
ANALISIS SINTACTICO INICIADO....
*****Analisis Sintactico terminado*****
0 error(es) encontrado(s).
```

Fuentes de Informacion

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2nd ed.). Pearson Education.

Appel, A. W. (2004). *Modern compiler implementation in Java* (2nd ed.). Cambridge University Press.

-oOo-